



C1

Recap → the modern AI stack → what's next

28 minutes

BY THE END OF THIS CAPSULE YOU WILL BE ABLE TO

Map any AI product as a stack of layers — and see what each layer does, how it fails, and what it costs.

1

Where we were — Class 1

RECAP

A 60-second recap before we build today's picture.

- 1 **AI is a taxonomy** — discriminative ML, generative, foundation models, agents, multimodal, physical AI.
- 2 **The jagged frontier** — capability doesn't track human difficulty — you test it.
- 3 **Sorting vs making** — discriminative sorts, generative makes; match the type to the problem.
- 4 **What scale changed** — foundation models; the two-file mental model; training vs inference.
- 5 **Case — Moderna** — democratising AI across an org, and the governance tension.
- 6 **You built** — a narrow model (scikit-learn) vs a foundation-model API on the same task.

Throughline: You made one API call. Today: everything wrapped around that call.

What you read before today

Five short pieces — each one seeds a capsule. Here's the through-line.

- 1 **a16z — Emerging Architectures** off-the-shelf layers (RAG), not trained → C1 · C3 · Colab
- 2 **Huyen — Designing ML Systems** the model is a small part; tie metrics to business value → C1 · C2 · C4
- 3 **Karpathy — Software 2.0** weights are code learned from data; data is the source → C1 · C4
- 4 **Husain — Your AI Product Needs Evals** an eval system, not a prompt, makes a product → C4 · Colab · A1
- 5 **DBS — AI-Powered Transformation** built the stack (ADA · ALAN · PURE), then GenAI on top → C1 · C2

Class 2 — objective & learning outcomes

FROM THE OBTL+ OUTLINE · WEEK 2

The modern AI stack — compute, models, fine-tuning, serving, vector databases, agent frameworks, observability and evaluation; build-vs-buy at each layer.

MAPS TO THESE LEARNING OUTCOMES

ILO 1

Distinguish AI types & select per problem — the stack, taxonomy, fine-tune vs retrieve.

ILO 2

Build an end-to-end AI solution (low-code + code) — today's RAG pipeline; assignment A1.

ILO 3

Evaluate design alternatives on cost, latency, risk vs accuracy — build-vs-buy, evals.

ILO 4 (communicate to a mixed audience) comes in lightly — the DBS case discussion.

Today: draw any AI product as a stack, defend build-vs-buy per layer, and build your first pipeline.

Today's plan

Where each piece happens — and roughly how long. One 15-min break, same for all three cohorts.

C1	The stack, end to end	~35 min	LECTURE · POLL
C2	Build-vs-buy + the DBS case	~55 min	CASE BRIEF · DISCUSSION
—	Break	20 min	BREAK
C3	Inside RAG — then build it (Colab)	~45 min	LECTURE · CODE
C4	Evals — how you know it works	~30 min	LECTURE · W3 PREVIEW

~3:05 plan + ~10 min buffer ≈ 3:15 · A1 releases Sun 16 Aug 2026 (end of Week 2), not in class.

From one API call to a product

Class 1 was the model. Class 2 is the system around it.

Class 1 — one call

```
response = model.generate("Answer this...")
```

- One request in, one answer out
- A great demo — not yet a product

Class 2 — the product around it

Seven layers turn a one-off answer into a dependable product.

- Compute + data — your content becomes vectors
- Vector store + model — find facts, then generate
- Orchestration + serving — glue it up, ship it fast
- Observability + evals — prove it actually works

Takeaway The model is one layer. The product is the whole stack.

The stack, end to end

Seven layers. We'll walk each — what it does, how it fails, what it costs.

- 1 Compute** GPUs / cloud that run everything
- 2 Data + embeddings** your content, turned into vectors
- 3 Vector store** indexes vectors; fast nearest-neighbour search
- 4 Model** the foundation model that generates
- 5 Orchestration** assembles prompts, calls, memory
- 6 Serving** delivers it fast, reliably, at scale
- 7 Observability + evals** traces, logs, metrics — plus tests that score quality

Underneath layer 1: the physical stack

Zoom into layer 1 — the GPUs (graphics processing units), interconnect and data centre that Compute rents.

- 1 Accelerator — training** thousands of GPUs in parallel for weeks; the one-off cost of building a model
- 2 Accelerator — inference** a few GPUs per request; a recurring cost that scales with users (slide 8)
- 3 Interconnect** NVLink inside a server, InfiniBand / RoCE across servers — caps training speed
- 4 Data centre** power, cooling and space — measured in megawatts, not servers
- 5 Supply + power** GPU allocation and grid capacity are the real 2026 bottlenecks

How each layer fails

Every layer has a distinct failure mode — that's what makes it a real layer.

Layer	How it fails
Compute	runs training / fine-tuning / serving — capacity + cost blow-ups, GPU scarcity
Data + embeddings	garbage in; stale or biased data
Vector store	wrong / irrelevant retrieval; won't scale
Model	hallucination; wrong model for the task
Orchestration	brittle prompt / chain logic
Serving	latency spikes; downtime under load
Observability + evals	silent failure — you don't know it broke

Takeaway If you can't name how a layer fails, you don't understand it yet.

What each layer costs

Illustrative ballparks, mid-2026 — prices fall fast, so treat these as directional.

Layer	Cost driver	Ballpark · mid-2026
Compute	GPU-hrs: train vs serve	H100 ≈ \$2–7 / GPU-hr; training ≈ \$10M+
Data + embeddings	embed API calls, per token + storage	≈ \$0.02 / 1M tokens · re-embed on change
Vector store	stored vectors + reads/writes	Pinecone ≈ \$50/mo + \$18 / 1M reads
Model (inference)	tokens per call (in + out)	≈ \$3 in / \$10 out per 1M; open ≈ \$0.3
Orchestration	engineering time	people, not per-call
Serving	kept-warm GPUs for low latency	≈ \$2–7/GPU-hr while warm, even idle — often cheaper cards
Evals + observability	human review + judge tokens	reviewer time + a little in tokens

Takeaway Training is a one-off; inference is a bill on every request — usually the biggest line.

Same stack, three industries

Read DOWN a column for one product; ACROSS a row to compare a layer. All seven layers, three businesses.

Layer	Retail · Lazada / Flipkart	Banking · DBS (our case)	Streaming · Netflix / Spotify
Compute	cloud GPU / CPU	private / on-prem cloud	GPU clusters (batch)
Data + embeddings	catalog → embeddings	ADA platform → embeddings	watch history → embeddings
Vector store	product-vector index	EKB vector store	item + user vectors
Model	ranking + LLM	GPT-4 / Claude (rented)	recommender
Orchestration	query → retrieve → rank	RAG over EKB	candidate-gen → rank
Serving	online, <100 ms	internal only, low volume	batch (nightly) + online
Observability + evals	search relevance / CTR	P-U-R-E governance	take-rate / A-B tests

Takeaway Learn the seven layers once — then read any AI product in any industry.

Two ways to use your own data

We go deep on this next capsule — and build it in the Colab.

Fine-tune

Update the model's weights on your examples.

- Full FT vs LoRA / PEFT adapters (cheap, swappable)
- Needs GPUs + curated labelled data
- Best for style / format / behaviour — not volatile facts

Retrieve (RAG)

Inject retrieved context at query time; weights stay fixed.

- embed → store → retrieve top-k → stuff prompt
- Update instantly — change the data; citable
- Bounded by context window + token cost

Takeaway Retrieve for knowledge, fine-tune for behaviour — most start with RAG (prompt-only before either).

POLL · WOOC LAP

Which layer would you never outsource for a bank?

Compute

Data

Vector store

Model

Serving

Evals / governance

Hold the answer — we'll test it against DBS in a minute.

POLL · WOOC LAP

Same question — for a fast-moving consumer AI startup?

Compute

Data

Vector store

Model

Serving

Evals / governance

Different answer? That's the point — 'own vs rent' depends on the business.

C1 · TAKEAWAY

A product is a stack of layers — each with a job, a failure mode, and a cost.

Next: which layers to build vs buy — then how retrieval actually works.

3

Coming up — Class 3

[NEXT](#)

Open the model, then steer it with prompts you can measure — a level up in depth from today (details firm up nearer the date).

- 1 **Inside the model** — tokens, attention, transformers, scaling laws — why capability emerges with scale.
- 2 **Prompting craft** — zero/few-shot, chain-of-thought, self-consistency, structured output.
- 3 **Eval-driven iteration** — build an eval set and improve the prompt against it — measure, don't guess.
- 4 **Token economics** — what a prompt really costs, and how to cut it.
- 5 **Hands-on (indicative)** — a step up from today — multi-step, structured-output prompting, iterated against your own evals.
- 6 **Milestone** — project problem statement — submit by Sun 23 Aug 2026, 11:59 PM.

Throughline: Prompting is programming in words — and evals are how you know it works.